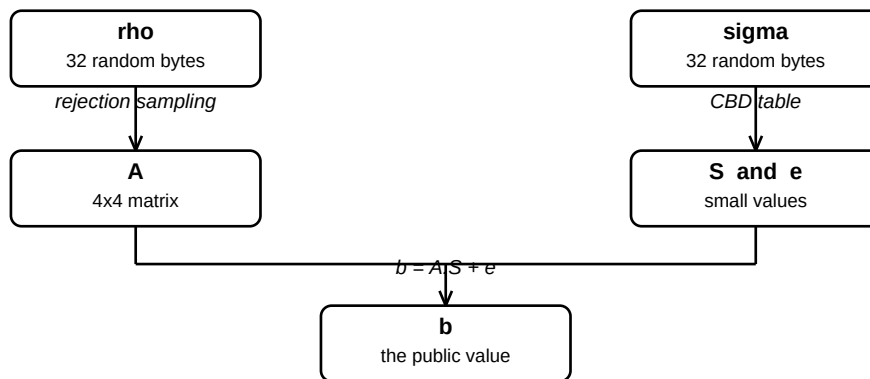


Act 1: Bob's KeyGen

Bob builds his key pair. Nothing has been sent yet and Alice does not exist as far as this Act is concerned. At the end of it Bob has something he can publish, and several things he must never let go of.



Separately, **zed** (32 random bytes) is generated and put straight in the drawer - it does nothing at all until Act 3, and only then if something has gone wrong.

Act 1 step by step

Step 1 - rho

```
rho = secrets.token_bytes(32)
```

WHAT: 32 random bytes.

HOW: secrets, not random - this has to be genuinely unguessable, not merely statistically random.

WHY IT MATTERS: this tiny seed gets shared **INSTEAD** of the whole matrix. A is 16 polynomials of 256 numbers each - 6144 bytes if you sent it raw. rho is 32 bytes. That is 192 times smaller, and anyone holding rho can rebuild A exactly.

Step 2 - A, the matrix

```
cell_seed = rho + bytes([row_num, col_num])
```

```
poly = poly_generator_from_seed(cell_seed)
```

WHAT: a 4x4 matrix where every cell is a full 256-coefficient polynomial.

HOW: for each cell, take rho and stick that cell's row and column on the end as a tag. Feed that tagged seed to the rejection sampler.

THE TRICK: one seed makes 16 completely different polynomials, because each cell gets a different tag. Cell (0,0) and cell (0,1) start from almost identical seeds, but hashing makes the two outputs completely unrelated.

Step 3 - zed

```
zed = secrets.token_bytes(32)
```

WHAT: 32 random bytes, Bob's hidden backup value.

HOW: same as rho.

WHY: absolutely nothing happens with it here. It waits in Bob's private key until Act 3, and only gets touched if a ciphertext check fails.

Step 4 - sigma, then S and e

```
sigma = secrets.token_bytes(32)
```

S uses positions 0,1,2,3

e carries on with positions 4,5,6,7

WHAT: S is Bob's private secret (4 polynomials), e is noise (4 polynomials).

HOW: the same tagging trick as A, but tagged with a plain counter instead of row and column. Uses shake_256 and the CBD table rather than rejection sampling, because these need to be TINY (-2 to +2), not spread over the whole range.

WORTH NOTICING: S takes positions 0 to 3, then e simply **CARRIES ON** counting from 4 to 7. One seed, one counter, 8 different polynomials, no second seed needed.

Act 1 step by step (continued)

Step 5 - $b = A.S + e$

```
dot_product = poly_add(dot_product, ntt_multiply(row[i], S[i]))
b.append(poly_add(dot_product, e[row_num]))
```

WHAT: the actual public key value.

HOW: a standard matrix times vector. For each row of A, multiply each entry by the matching entry of S, add all four together, then add that row's noise from e.

WHY THIS IS THE ENTIRE SECURITY: given A and b, working backwards to find S is the hard lattice problem. Without the noise e it would be ordinary school algebra and anyone could solve it. The noise is what makes it hard - and it is small enough that the message still decodes correctly at the other end.

Step 6 - $H(ek)$

```
b_encoded = b''.join(byte_encode(poly, 12) for poly in b)
combined_bytes = rho + b_encoded
H(ek) = sha3_256(combined_bytes)
```

WHAT: a 32-byte fingerprint of Bob's own public key.

HOW: byte-encode b at 12 bits per coefficient, stick rho on the front, hash once.

WHY: worked out once and kept. Alice independently computes the identical value in Act 2, and Bob reuses his stored copy in Act 3 rather than recomputing it.

End of Act 1 - what is public and what is not

PUBLIC - this is (rho, b)

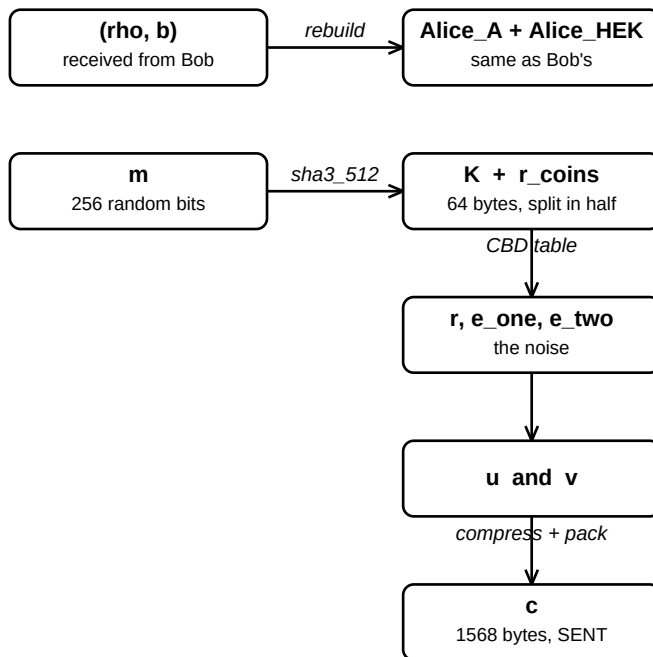
rho (32 bytes)
b (4 polynomials)

PRIVATE - never leaves Bob

S - the secret
e - the noise
sigma - the seed they came from
zed - the backup
H(ek) - the stored fingerprint

Act 2: Alice's Encapsulate

Alice receives (rho, b) and nothing else. From that alone she produces two things:
a shared secret only she knows so far, and one ciphertext to send back.



K never moves. It stays on Alice's side. Only c is sent.

Act 2 step by step

Step 1 - Alice_A

WHAT: Alice rebuilds Bob's matrix herself.

HOW: identical loop to Act 1 - same rho, same row and column tags, same rejection sampler. She was never sent the matrix, only the seed it came from.

Step 2 - Alice_HEK

WHAT: Alice works out the same public-key fingerprint Bob already stored.

HOW: byte-encode b, stick rho on the front, sha3_256. Exactly what Bob did.

WHY: it binds everything that follows to this specific public key. Without it, the shared secret would not be tied to this particular exchange.

Step 3 - m, the message

```
m.append(secrets.choice([0, 1])) x 256
```

WHAT: 256 random bits. This is the ONLY genuinely fresh randomness in Act 2.

HOW: secrets again, because everything downstream is derived from this.

WORTH KNOWING: m is not a message in the ordinary sense. Nobody wants to read it. Its whole job is to be random and to be recoverable by Bob later.

Step 4 - K and r_coins

```
m_bytes = byte_encode(m, 1)          -> 32 bytes
g_output = sha3_512(m_bytes + Alice_HEK) -> 64 bytes
K         = g_output[0:32]
r_coins   = g_output[32:64]
```

WHAT: one hash, cut clean down the middle.

HOW: pack m into bytes (d=1, so 8 bits per byte), glue Alice_HEK on, hash once with sha3_512 which always gives back exactly 64 bytes, then split.

K - the real shared secret. Alice sets it aside and sends it nowhere.

r_coins - a seed, used immediately to make the noise for this encryption.

THIS IS THE CLEVER PART: the noise is DERIVED from the message rather than picked independently. That is what lets Bob later redo the whole thing from scratch and check that the ciphertext was built honestly.

Act 2 step by step (continued)

Step 5 - r, e_one, e_two

```
r      takes positions 0..3
e_one carries on 4..7
e_two takes position 8
```

WHAT: the noise for this one encryption.

HOW: the same one-seed-plus-counter trick used for S and e in Act 1, just seeded from r_coins instead of sigma. Same shake_256 and CBD table.

Step 6 - m_encoded

```
m_encoded.append(((q + 1) // 2) * m[i])
```

WHAT: turns each bit of m into a real number - either 0 or 1665.

HOW: q is 3329, an odd number, so half of it is 1664.5. Rounding to nearest gives 1665, which is what (q+1)//2 produces.

WHY 1665 AND NOT 0 AND 1: spreading the two possible values as far apart as possible gives the noise maximum room to wobble without ever flipping the answer.

Step 7 - A_t and b_t

A_t is the transpose of Alice_A - rows become columns.

b_t is literally just b. Transposing a vector does nothing, so it is a no-op.

Step 8 - u and v

```
u = A_t x r + e_one
v = b_t . r + e_two + m_encoded
```

WHAT: the two mathematical values that carry the message.

HOW: same matrix and vector maths as Act 1, using ntt_multiply.

u - carries no message, it is what lets Bob undo the maths using his secret S.

v - carries the actual message, hidden underneath noise and the b.r term.

Step 9 - c, the ciphertext

```
c = ciphertext_maker(u, v)
```

WHAT: the single blob that actually gets sent.

HOW: compress u to 11 bits and v to 5 bits, byte-encode both, glue together.

1408 + 160 = 1568 bytes, matching FIPS 203 Table 3 exactly.

END OF ACT 2: Alice holds K privately. Bob has been sent c and nothing else.

Neither the key nor the message ever travelled.